Historical Candle Data V3

`GET` `/historical-candle/:instrument_key/:unit/:interval/:to_date/:from_date`

Compared to the standard Historical Candle Data API, API V3 allows users to retrieve data in custom time intervals for each unit (`minutes`, `hours`, `days`, `weeks`, and `months`), enabling more granular data control and improved flexibility for analysis. The response structure maintains consistency with the standard Historical Candle Data API format, facilitating seamless integration with existing applications and systems.

The API is designed to handle large volumes of data efficiently, ensuring quick response times even for extensive historical queries.

This API is particularly useful for traders and analysts who need to analyze historical price movements and trends in the financial markets.

The API offers OHLC (Open, High, Low, Close) data for instruments across multiple timeframes, with different historical availability and retrieval constraints based on the selected unit and interval as per the below table:

Unit	Interval Options	Historical Availability	Max retrieval record limit
minutes	1, 2, ... 300	Available from January, 2022	`1 month` - Applicable for intervals ranging from 1 to 15 minutes. `1 quarter` - Used for intervals greater than 15 minutes, up to the specified to_date.
hours	1, 2, ... 5	Available from January, 2022	`1 quarter` leading up to the to_date.
days	1	Available from January, 2000	`1 decade` leading up to the to_date.
weeks	1	Available from January, 2000	No limit.
months	1	Available from January, 2000	No limit.

New Instruments

- **Global Index** — Major global stock market indices such as GIFT NIFTY, Dow Jones, S&P, FTSE 100, and more. See Global Instruments for details and download the Global Instruments file for instrument keys.
- **Global Indicators** — Economic and commodity indicators including USD INR, Oil (Brent), and Oil (WTI). See Global Instruments for details and download the Global Instruments file for instrument keys.
- **India VIX** — The NSE Volatility Index, available using instrument key `NSE_INDEX|India VIX`.

Request

```
curl --location 'https://api.upstox.com/v3/historical-
candle/NSE_EQ%7CINE848E01016/minutes/1/2025-01-02/2025-01-01' \
--header 'Content-Type: application/json' \
--header 'Accept: application/json' \
--header 'Authorization: Bearer {your_access_token}'
```

Additional samples in various languages are available in the Sample Code section on this page.

Path Parameters

instrument_key

REQUIRED

Required

TYPE

`string`

DESCRIPTION

The unique identifier for the financial instrument for which historical data is being queried. For the regex pattern applicable to this field, see the Field Pattern Appendix.

unit

REQUIRED

Required

TYPE

`string`

DESCRIPTION

Specifies the unit for the candles.
Possible values: `minutes`, `hours`, `days`, `weeks`, `months`.

interval

REQUIRED

Required

TYPE

`string`

DESCRIPTION

Specifies the interval for the candles.
Possible values:
`1`, `2`, `3`, ..., `300` for minutes.
`1`, `2`, ..., `5` for hours.
`1` for days, weeks, and months.

to_date

REQUIRED

Required

TYPE

`string`

DESCRIPTION

The ending date (inclusive) for the historical data range. Format: 'YYYY-MM-DD'.

from_date

REQUIRED

Optional

TYPE

`string`

DESCRIPTION

The starting date for the historical data range. Format: 'YYYY-MM-DD'.

Responses

200 4XX

Response Body


```
{
  "status": "success",
  "data": {
    "candles": [
      [
        "2025-01-01T00:00:00+05:30",
        53.1,
        53.95,
        51.6,
        52.05,
        235519861,
        0
      ],
      [
        "2025-02-01T00:00:00+05:30",
```

```
        50.35,
        56.85,
        49.35,
        52.8,
        1004998611,
        0
      ]
```

status

TYPE

`string`

DESCRIPTION

A string indicating the outcome of the request. Typically `success` for successful operations.

data

TYPE

`object`

DESCRIPTION

Contains OHLC values for all instruments across various timeframes.

data.candles

TYPE

`array[]`

DESCRIPTION

Array of candle data, each presented as an array with sequential elements representing trading activity.

data.candle[0]

TYPE

`number`

DESCRIPTION

`Timestamp`: Indicating the start time of the candle's timeframe.

data.candle[1]

TYPE

`number`

DESCRIPTION

`Open`: The opening price of the asset for the given timeframe.

data.candle[2]

TYPE

`number`

DESCRIPTION

`High`: The highest price at which the asset traded during the timeframe.

data.candle[3]

TYPE

`number`

DESCRIPTION

`Low`: The lowest price at which the asset traded during the timeframe.

data.candle[4]

TYPE

`number`

DESCRIPTION

`Close`: The closing price of the asset for the given timeframe.

data.candle[5]

TYPE

`number`

DESCRIPTION

`Volume`: The total amount of the asset that was traded during the timeframe.

data.candle[6]

TYPE

`number`

DESCRIPTION

`Open Interest`: The total number of outstanding derivative contracts, such as options or futures.

Sample Code

Get data with a 1-minute interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/minutes/1/2025-01-02/2025-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)
```

```python
# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a 3-minute interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/minutes/3/2025-01-02/2025-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a 15-minute interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/minutes/15/2025-01-04/2025-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
```

```
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a 1-hour interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/hours/1/2025-02-01/2025-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a 4-hour interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/hours/4/2025-02-01/2025-01-01'
```

```python
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a daily interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/days/1/2025-03-01/2025-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a weekly interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests
```

```python
url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/weeks/1/2025-01-01/2024-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

Get data with a monthly interval

Curl | Python | Node.js | Java | PHP | Python SDK | Node.js SDK | Java SDK

```python
import requests

url = 'https://api.upstox.com/v3/historical-candle/NSE_EQ%7CINE848E01016/months/1/2025-01-01/2010-01-01'
headers = {
    'Content-Type': 'application/json',
    'Accept': 'application/json',
    'Authorization': 'Bearer {your_access_token}'
}

response = requests.get(url, headers=headers)

# Check the response status
if response.status_code == 200:
    # Do something with the response data (e.g., print it)
    print(response.json())
else:
    # Print an error message if the request was not successful
    print(f"Error: {response.status_code} - {response.text}")
```

NOTE

- Use the Intraday Candle Data API V3 to retrieve data specific to the current trading day with the wide range of units and intervals
- It's important to note that if the retrieval record limit is exceeded, then the API will throw an error indicating that the request is invalid.
- This V3 API provides additional custom interval options that are not available in the standard Historical Candle Data API